



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



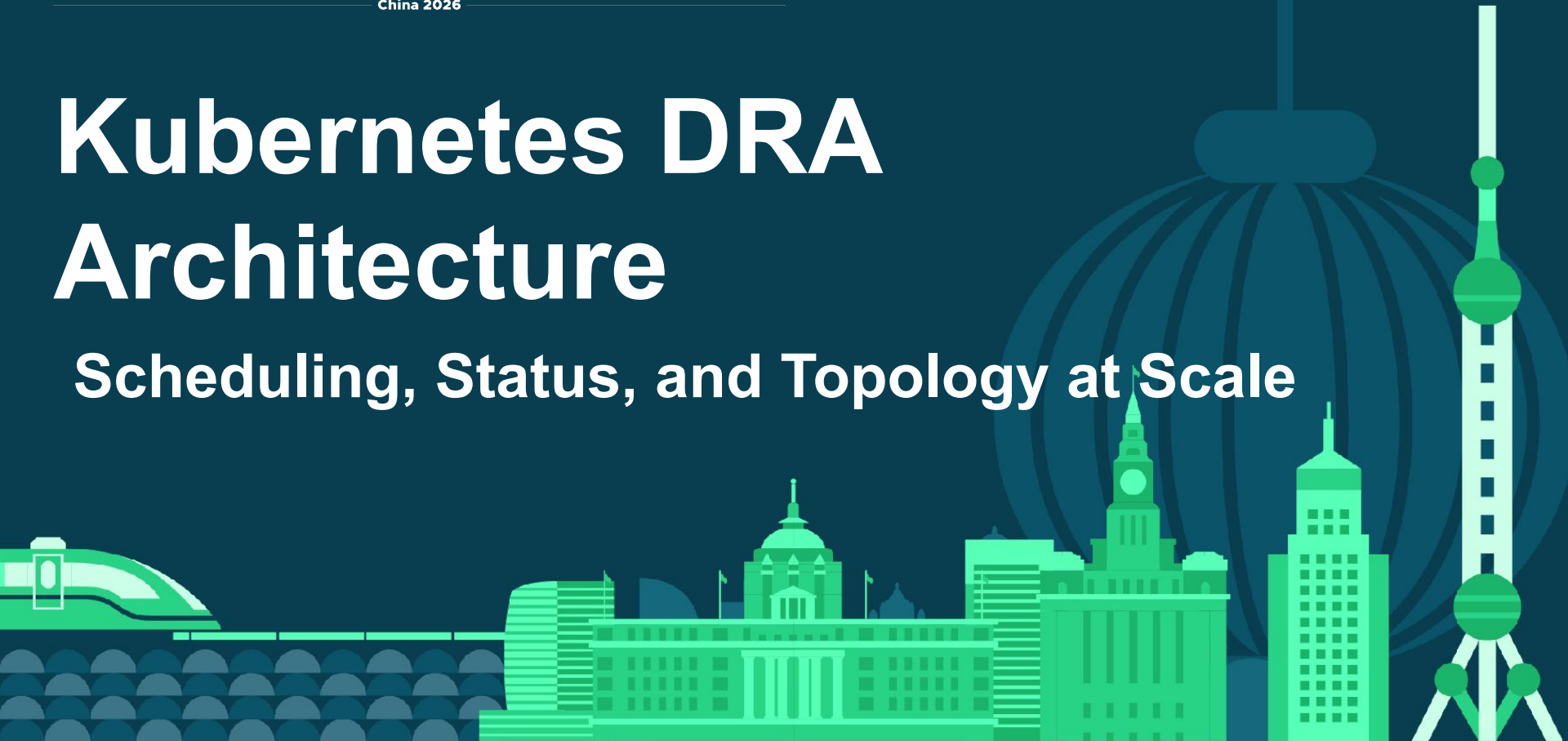
PyTorch
CONFERENCE

#KubeCon #CloudNativeCon
#OpenInfraSummit #PyTorchCon

China 2026

Kubernetes DRA Architecture

Scheduling, Status, and Topology at Scale



Intro us



China 2026

Paco Xu (@pacoxu)



Open Source Team Leader at DaoCloud

- Kubernetes Steering Committee
- Kubeadm Maintainer
- CNCF Ambassadors
- LFAPAC Evangelist



DaoCloud

Kang Zhang (@kangclzjc)



Senior Solution Architect at NVIDIA

- Grove Maintainer
- Dynamo Planner Maintainer
- Koordinator Member



NVIDIA



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

China 2026

Kubernetes DRA Update



Principles That Turn DRA from an API into a System

01

Resources Must Be Structured

Attributes, capacity, and accessible nodes must become scheduling facts.

02

Topology Must Inform Decisions

PodGroup placement and device allocation must share the same constraint space.

03

State Must Align with Topology

Ownership boundaries determine the conflict domain and convergence speed at thousand-GPU scale.

DRA is a resource model, a scheduling protocol, and a state boundary—not a GPU switch.



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

DRA update 1.32–1.37



China 2026

KEP	Description	1.32	1.33	1.34	1.35	1.36	1.37
4381	DRA: Structured Parameters	Beta	Beta	Stable	Stable	Stable	Stable
5004	DRA: Extended Resource Requests via DRA	—	—	Alpha	Alpha	Beta	• Stable
4817	DRA: ResourceClaim Device Status	Alpha	Beta	Beta	Beta	Beta	• Stable
5018	DRA: Namespace-controlled Admin Access	—	Alpha	Beta	Beta	Stable	Stable
5055	DRA: Device Taints and Tolerations	—	Alpha	Alpha	Alpha	Beta	• Stable
4816	DRA: Prioritized Alternatives in Device Requests	—	Alpha	Beta	Beta	Stable	Stable
5075	DRA: Consumable Capacity	—	—	Alpha	Alpha	Beta	Beta
4815	DRA: Partitionable Devices	—	Alpha	Alpha	Alpha	Beta	Beta
5304	DRA: Device Attributes in Downward API	—	—	—	—	Alpha	• Beta
5729	DRA: ResourceClaim Support for Workloads	—	—	—	—	Alpha	• Beta
4680	Resource Health Status in Pod Status	Alpha	Alpha	Alpha	Alpha	Beta	Beta
5491	DRA: List Types for Attributes	—	—	—	—	Alpha	Alpha†
6132	PreQueueingHint Ext Point for Event Processing	—	—	—	—	—	• Alpha
5517	DRA: Node Allocatable Resource Requests	—	—	—	—	Alpha	• Alpha 2
5677	DRA: Resource Availability Visibility	—	—	—	—	Alpha	• Alpha
5007	DRA: Device Binding Conditions	—	—	Alpha	Alpha	Beta	Beta
5945	DRA: Optional Node Preparation	—	—	—	—	—	• Alpha
5963	DRA: Device Compatibility Groups	—	—	—	—	—	• Alpha
6072	DRA: Standard numaNode Device Attribute	—	—	—	—	—	• Stable
6080	DRA: Derived Attributes	—	—	—	—	—	• Alpha

• tracked for v1.37 | Alpha 2 = second Alpha iteration | † removed from v1.37 at Code Freeze

DRA Planning for 1.38



China 2026

Tentative planning based on past discussions.

Pri	KEP	Description	Owner SIG	Lead	Last Releases		Next	Notes
		See also the DRA project board .			1.36	1.37	1.38	
P1	5075	DRA: Consumable Capacity	Scheduling	sunya-ch	Beta	Beta	<i>Stable</i>	
		Fractional Consumable Capacity	Scheduling	sunya-ch		Beta	<i>Stable</i>	off by default in 1.37, on if downgrades are disabled (MinCompatibilityVersion 1.37)
P1	4815	DRA: Partitionable Devices	Scheduling	mortent	Beta	Beta	<i>Stable</i>	
P1	6132	Scheduler: PreQueueingHint	Scheduling	geetasg, bgee		Alpha	<i>Beta</i>	Addresses “thundering herd” of pods with per-pod claims
P2	4680	Resource Health Status in Pod Status	Node	harche	Beta	Beta	<i>Beta</i>	Stay in beta, gather feedback
P1	5304	DRA: Attributes Downward API	Node	alaypatel07	Alpha	Beta	<i>Stable</i>	Supports injecting devices into KubeVirt VM
P1	5729	DRA: ResourceClaim Support for Workloads	Scheduling	nojnjuh	Alpha	Beta	<i>Beta</i>	
P1	5517	DRA: Node Allocatable Resource Requests	Scheduling	pravk03	Alpha	Alpha	<i>Beta</i>	
P1	5677	DRA: Resource Availability Visibility	Node	nmn3m	Alpha	Alpha	<i>Beta</i>	UX improvement
P2	5007	DRA: Device Binding Conditions	Scheduling	ttuuubasa	Beta	Beta	<i>Stable</i>	
P2	5491	DRA: List Types for Attributes	Scheduling	bg-chun, everpeace	Alpha	Alpha	<i>Beta</i>	More flexible hardware topologies
P2	5941	DRA: Shared Consumable Capacity	Scheduling	SchSeba, LionelJouin			<i>Alpha</i>	Missed 1.37; target 1.38
P1	5690	DRA: preemption	Scheduling	mortent			<i>Beta</i>	Deferred to 1.38
P2	5945	DRA: Optional Node Preparation	Node	troychiu		Alpha	<i>Beta</i>	
P1	5963	DRA Device Compatibility Groups	Scheduling	omeryahud		Alpha	<i>Beta</i>	Closes a shortcoming of partitionable devices.
P2	6080	DRA: Derived Attributes	Scheduling	gauravghildiyal		Alpha	<i>Beta</i>	
P2	5981	DRA: Sharing Affinity for Consumable Capacity	Scheduling	ashvindeodhar			<i>Alpha</i>	Missed 1.37; target 1.38

Tentative planning based on past discussions.

*Note: SIG scheduling wants to better align WAS and DRA in 1.38. That may means **some of these need to pause** while we move some DRA concepts from the plugin to the scheduling framework.*

Pri	KEP	Description	Owner SIG	Lead	Last Releases		Next	Notes
		See also the DRA project board .			1.36	1.37	1.38	
P1	5075	DRA: Consumable Capacity	Scheduling	sunya-ch	Beta	Beta	<i>Stable</i>	
		Fractional Consumable Capacity	Scheduling	sunya-ch		Beta	<i>Stable</i>	off by default in 1.37, on if downgrades are disabled (MinCompatibiltyVersion 1.37)
P1	4815	DRA: Partitionable Devices	Scheduling	mortent	Beta	Beta	<i>Stable</i>	
P1	6132	Scheduler: PreQueueingHint	Scheduling	geetasg, bgee		Alpha	<i>Beta</i>	Addresses “thundering herd” of pods with per-pod claims
P2	4680	Resource Health Status in Pod Status	Node	harche	Beta	Beta	<i>Beta</i>	Stay in beta, gather feedback
P1	5304	DRA: Attributes Downward API	Node	alaypatel07	Alpha	Beta	<i>Stable</i>	Supports injecting devices into KubeVirt VM
P1	5729	DRA: ResourceClaim Support for Workloads	Scheduling	nojn huh	Alpha	Beta	<i>Beta</i>	
P1	5517	DRA: Node Allocatable Resource Requests	Scheduling	pravk03	Alpha	Alpha	<i>Beta</i>	
P1	5677	DRA: Resource Availability Visibility	Node	nmn3m	Alpha	Alpha	<i>Beta</i>	UX improvement
P2	5007	DRA: Device Binding Conditions	Scheduling	ttuuubasa	Beta	Beta	<i>Stable</i>	
P2	5491	DRA: List Types for Attributes	Scheduling	bg-chun, everpeace	Alpha	Alpha	<i>Beta</i>	More flexible hardware topologies
P2	5941	DRA: Shared Consumable Capacity	Scheduling	SchSeba, LionelJouin			<i>Alpha</i>	Missed 1.37; target 1.38
P1	5690	DRA: preemption	Scheduling	mortent			<i>Beta</i>	Deferred to 1.38
P2	5945	DRA: Optional Node Preparation	Node	troychiu		Alpha	<i>Beta</i>	
P1	5963	DRA Device Compatibility Groups	Scheduling	omeryahud		Alpha	<i>Beta</i>	Closes a shortcoming of partitionable devices.
P2	6080	DRA: Derived Attributes	Scheduling	gauravghildiyal		Alpha	<i>Beta</i>	
P2	5981	DRA: Sharing Affinity for Consumable Capacity	Scheduling	ashvindeodhar			<i>Alpha</i>	Missed 1.37; target 1.38

Kubernetes v1.37 DRA KEP Roadmap

Turning v1.37 DRA updates into an end-to-end journey — with release tracking made explicit

GA

Beta

Alpha

• Formally tracked for v1.37



Foundation / Core KEP-4381 DRA Core / Structured Parameters

The stable DRA API and structured allocation contract

GA

01 Request Intake

• **KEP-5004**

GA

Extended Resource Support

Legacy extended-resource requests can be served by DRA drivers.

• **KEP-5729**

Beta

ResourceClaim Support for Workloads

Workload / PodGroup objects can reference shared ResourceClaims.



02 Resource Modeling

KEP-4815

Beta

Partitionable Devices
Mutually exclusive device partitions

KEP-5075

Beta

Consumable Capacity
Share capacity across independent claims

KEP-5491

Alpha

List Types for Attributes
Set semantics for topology attributes

• **KEP-5517**

Alpha

Node Allocatable Resources
Expose CPU / memory allocatable to DRA

• **KEP-6080**

Alpha

Derived Attributes
Virtual grouping keys from per-device CEL

• **KEP-6072**

GA

Standard NUMA Node Attribute
resource.kubernetes.io/numaNode
for cross-driver co-placement

03 Scheduling & Binding

KEP-4816

GA

Prioritized List

Priority-ordered device request alternatives

KEP-5007

Beta

Device Binding Conditions

Attach / prepare before binding completes

• **KEP-5055**

GA

Device Taints & Tolerations

Device health isolation & maintenance

• **KEP-5963**

Alpha

Device Compatibility Groups

Reject incompatible device co-allocation

KEP-6132

Alpha

PreQueueingHint Extension Point

DRA claim queue: $O(N^2) \rightarrow O(1)$
~2.4x · SchedulerPreQueueingHints

04 Node Execution



kubelet DRA plugin

NodePrepareResources

CDI / Runtime

• **KEP-5304**

Beta

Device Metadata in Containers

Mount versioned device metadata
into workload containers.

• **KEP-5945**

Alpha

Optional Node Preparation

Skip kubelet node operations
when a device does not require them.

05 Status / Ops / Observability

• **KEP-4817**

GA

ResourceClaim Device Status

Claim status reports allocated device state

KEP-4680

Beta

Resource Health Status

Pod status exposes allocated-device health

KEP-3695

GA

PodResources API for DRA

Per-node query of DRA allocation results

• **KEP-5677**

Alpha

Resource Availability Visibility

Query ResourceSlice pool availability

KEP-5018

GA

DRA AdminAccess

Secure administrative access to
allocated devices

ResourceSlice /
DeviceClass

ResourceClaim



Scheduler



Claim.status



NodePrepareResources



CDI / Runtime

Pod /
Observability

Which device to use, how to use it, and whether it is ready?



China 2026

Device Plugin

“How many are available?”

Node Allocatable

`vendor.com/device: N`

Scheduler fits workloads based on integer counts

kubelet Allocate / runtime injection

Homogeneous · Node-local · Quantity-based

Dynamic Resource Allocation

“Which one, how should it be used, and is it ready?”

ResourceSlice publishes device facts

ResourceClaim expresses identity, capacity, and constraints

Scheduler selects a specific combination of devices

Driver prepares devices; reports status and health

Structured · Partitionable · Observable · Governable

Fully Unlocking GPU Compute Efficiency



China 2026

Challenge: GPUs Are Expensive but Underutilized

Shift from “competing for GPUs” to compute scheduling, queue governance, and heterogeneous resource sharing

- Training, fine-tuning, inference, offline workloads, development, and testing all compete for GPUs. The traditional model of simply “requesting a number of GPUs” is no longer sufficient.

Scheduling

DRA / TAS

- Attribute-based device requests
- Topology-aware scheduling

DRANET (initiated by Google)
improves bandwidth utilization by

60%

Batch & Workloads

Kueue / Volcano
Queues, priorities, and quotas
Workload colocation,
preemption, and fair sharing

Sharing & Heterogeneity

HAMi
Sharing, isolation, and
scheduling Unified support for
GPUs, TPU and etc.

DaoCloud vGPU reduces
operating costs by ~ **50%**

Virtualization

KWOK Mocking(Almost no cost)
NVIDIA AICR
Ai-configurator
Reduce verification and delivery
costs

Note: some new-generation GPUs are supported exclusively through the DRA driver.

The goal is not merely to “run whenever GPUs are available,” but to make GPUs platform resources that can be queued, quota-managed, shared, and operated efficiently.

KEP-5729 ResourceClaim Support for Workloads



China 2026

Multiple Pods in a PodGroup share a single templated ResourceClaim.

Beta

The PodGroup creates the Claim. Each member Pod references it using the same logical name, and its status resolves that name to the same concrete Claim.

1 PodGroup declares the shared template

```
apiVersion: scheduling.k8s.io/v1beta1
kind: PodGroup
metadata:
  name: train-job
spec:
  resourceClaims:
  - name: model-cache
    resourceClaimTemplateName:
      model-cache-template
  minMember: 2
```

2 Member Pods reference the logical name

```
apiVersion: v1
kind: Pod
metadata:
  name: worker-0
spec:
  schedulingGroup:
    podGroupName: train-job
  resourceClaims:
  - name: model-cache
    resourceClaimTemplateName:
      model-cache-template
  containers:
  - name: worker
    image: example/train:v4
    resources:
      claims:
      - name: model-cache
  ---
# worker-1 uses the same claim
```

3 PodGroup, Pod, and Claim

```
# PodGroup status
resourceClaimStatuses:
- name: model-cache
  resourceClaimName:
    train-job-model-cache-7db2
---
# Pod status
resourceClaimStatuses:
- name: model-cache
  resourceClaimName:
    train-job-model-cache-7db2
---
# ResourceClaim status
reservedFor:
- apiGroup: scheduling.k8s.io
  resource: podgroups
  name: train-job
  uid: 0d9c...
```

Conclusion

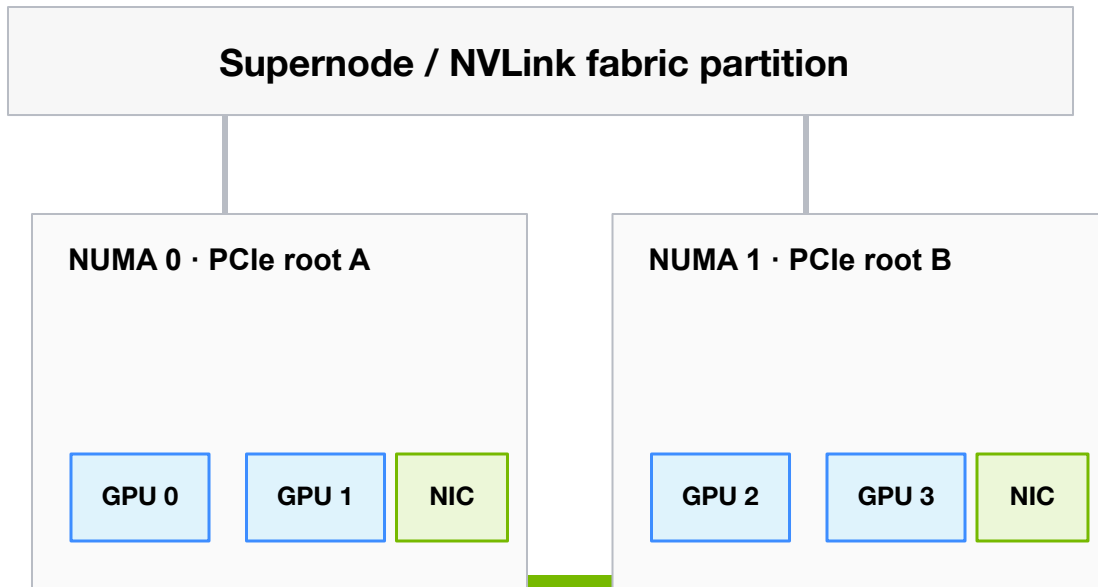
The concrete Claim is managed through the PodGroup lifecycle, while Pods hold only its logical name. The PodGroup and Pod statuses can be cross-checked against Claim.reservedFor.

v1.37 Beta
DRAWorkloadResourceClaims +
GenericWorkload (Beta)

Supernode: NUMA, PCIe, and Fabric Topology



China 2026



ResourceClaimTemplate.spec(example)

```
devices:
  requests:
    - name: gpu
      exactly: {deviceClassName: gpu, count: 2}
    - name: nic
      exactly: {deviceClassName: rdma-nic, count: 1}
  constraints:
    - requests: ["gpu", "nic"]
      matchAttribute:
        topology.example.com/pcieRoot
```

matchAttribute brings the “same PCIe root” constraint into device combination selection for a single claim.

KEP-6080 Derived Attributes



China 2026

Derived Attributes normalize vendor-specific attribute names within a Claim.

Alpha

The GPU driver exposes numa, while the NIC driver exposes numaNode. Each request derives a common shared-numa-node attribute using CEL, then uses matchAttribute to co-locate the devices.

1 Different drivers use different attribute schemas

```
apiVersion: resource.k8s.io/v1
kind: ResourceSlice
metadata: {name: node-a-gpu}
spec:
  driver: gpu.nvidia.com
  pool: {name: node-a, generation: 1,
resourceSliceCount: 1}
  nodeName: node-a
  devices:
    - name: gpu-0
      basic:
        attributes:
          gpu.nvidia.com/numa: {int: 0}
---
apiVersion: resource.k8s.io/v1
kind: ResourceSlice
metadata: {name: node-a-nic}
spec:
  driver: dra.net
  pool: {name: node-a, generation: 1,
resourceSliceCount: 1}
  nodeName: node-a
  devices:
    - name: nic-3
      basic:
        attributes:
          dra.net/numaNode: {int: 0}
    - name: nic-4
      basic:
        attributes:
          dra.net/numaNode: {int: 1}
```

2 Each request derives a normalized attribute

```
spec:
  devices:
    requests:
      - name: gpu
        exactly:
          deviceClassName: gpu.example.com
          derivedAttributes:
            - name: shared-numa-node
              expression: |
                device.attributes[
                  "gpu.nvidia.com"].numa
      - name: nic
        exactly:
          deviceClassName: nic.example.com
          derivedAttributes:
            - name: shared-numa-node
              expression: |
                device.attributes[
                  "dra.net"].numaNode
  constraints:
    - requests: [gpu, nic]
      matchAttribute: shared-numa-node
```

3 Status: devices allocated on the same NUMA node

```
status:
  allocation:
    devices:
      results:
        - request: gpu
          driver: gpu.example.com
          pool: node-a
          device: gpu-0
        - request: nic
          driver: nic.example.com
          pool: node-a
          device: nic-3
---
# Derived values participate only in scheduling:
# gpu-0 → shared-numa-node = 0
# nic-3 → shared-numa-node = 0
# nic-4 → shared-numa-node = 1(no selected)
```

v1.38 DRA plan



China 2026

KEP	Title	Target	Planned work
5075	DRA: Consumable Capacity	Stable	GA validation, real-world adoption, and scheduler performance
5304	DRA: Device Attributes in Downward API	Stable	Production validation of device metadata delivery through CDI
5517	DRA: Node Allocatable Resource Requests	Beta	Eviction, quota, LimitRange, and ephemeral-storage integration
5945	DRA: Optional Node Preparation	Beta	Default enablement, upgrade/rollback tests, and scale validation
5963	DRA: Device Compatibility Groups	Beta	Production-driver validation of compatibility groups
6080	DRA: Derived Attributes	Beta	Driver feedback, E2E coverage, and scheduler performance
4680	Resource Health Status in Pod Status	Stable?	Real-world health-reporting adoption and GA graduation
5941	DRA: Shared Consumable Capacity	Alpha†	Deferred implementation of shared request-driven capacity

Note: These are tentative plans and may change based on implementation progress.



China 2026

Updates Beyond Kubernetes



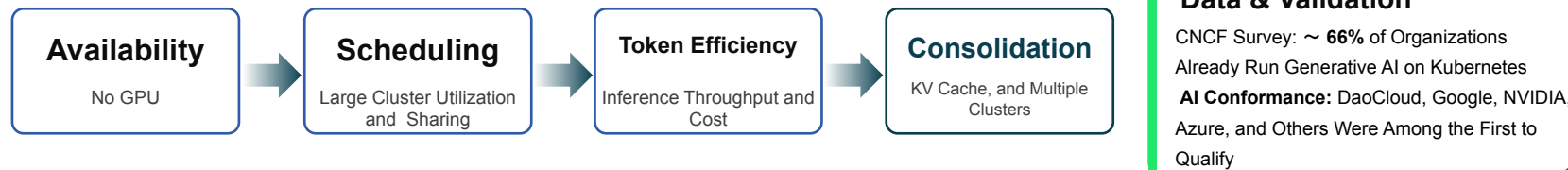
AI Infrastructure Is Entering the Cloud-Native Era



China 2026

CNCF, part of the Linux Foundation, believes that cloud native is evolving from an application runtime foundation into the foundation for engineering, scaling, and governing AI.

AI 的基础操作系统正收敛到 Kubernetes



Trends



- 1 Google GKE / Anthropic:** Supported 250k GPUs in 2024; 130k Nodes in a Single Cluster in 2025; Hypercluster to Exceed 1 Million GPUs in 2026
- 2 AWS EKS / Anthropic / Nova:** 100k Nodes in a Single Cluster; 800k NVIDIA GPUs / 1.6 Million Trainium Accelerators
- 3 ByteDance:** More Than 500k Nodes Across Multiple Clusters; 20k Nodes in a Single Cluster; More Than 1M GPUs
- 4 Ant Group / Alibaba:** Kubernetes Deployments Exceeding 20,000 Nodes; Unified Training and Inference Scheduling; GPU Pooling Across 500,k GPUs
- 5 DeepSeek / High-Flyer:** HAI Platform Built on Kubernetes; Fire-Flyer 2 with 10k GPUs; V3 Trained on 2,048 GPUs
- 6 Meituan LongCat-2.0:** A 1.6-Trillion-Parameter MoE Model; Built Using a Cluster of 50k Domestically Produced Accelerators

The Next Challenge: How to Unify GPUs, Workloads, Inference, Caching, and Multi-Cluster Resources.

DRA Is Becoming the Unified Entry Point for AI Resource Management

- Secure accelerator access (MUST)
- High-performance **networking** (SHOULD)
 - For specialized pod-to-pod networking, resources should be discoverable, schedulable and multi-interface capable—using DRA / DRANET.
- Driver and runtime verification (SHOULD)
- Static and dynamic GPU sharing (SHOULD)
- Virtualized accelerators (vGPU) (SHOULD)

Either Device Plugin or DRA can satisfy the requirement.

No requirement makes DRA the only compliant path.

Ecosystem Co-Evolution



China 2026

DRA Drivers in Kubernetes SIGs

[dra-driver-nvidia-gpu](#), [dranet](#), [dra-driver-cpu](#), [cni-dra-driver](#), [dra-driver-google-tpu](#), [dra-driver-topology](#)

Ecosystem

Kueue	Admission and quota management; actual device allocation is still handled by the scheduler and DRA
KAI / Volcano	Provide additional batch, gang, and topology-aware scheduling capabilities
Native scheduler	The long-term standard path; WAS/TAS and some DRA capabilities are still evolving
HAMi	Fractional device sharing and runtime isolation; DRA handles allocation while HAMi enforces resource limits



China 2026

dra-driver-nvidia-gpu

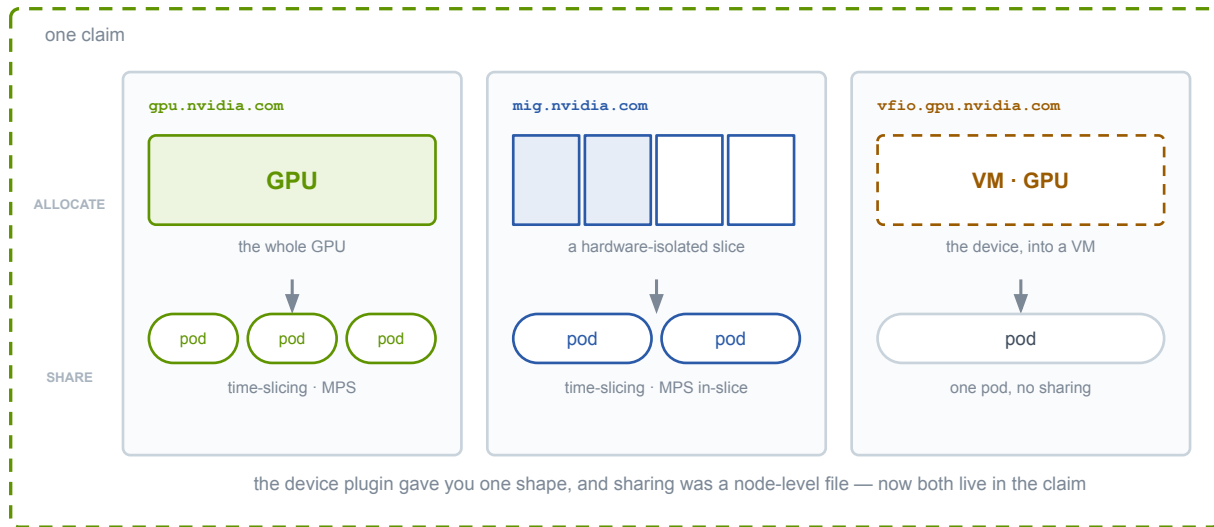


GPU allocation



China 2026

Part 1



gpu.nvidia.com

the DRA replacement for the `nvidia.com/gpu` path

DeviceClasses: `gpu · mig · vfio.gpu` (`.nvidia.com`)

officially supported

DESIGN GOAL

Move selection, sharing and configuration out of node-level files and node labels, and into the claim next to the workload.

AXES · What you allocate is a DeviceClass · how it is shared is opaque config on the claim — time-slicing and MPS are alpha and off by default

■ One subsystem, one question: how does a pod ask for a GPU now?

From nvidia.com/gpu to ResourceClaims

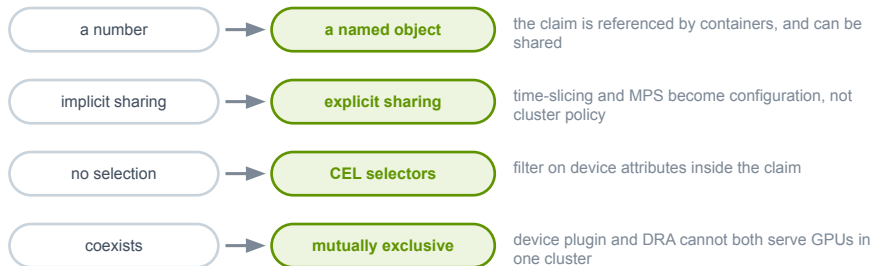


China 2026

BEFORE · device plugin

```
resources:
  limits:
    nvidia.com/gpu: 1
```

one object, one integer, repeated on every pod



RULE · Using the device plugin and the DRA driver for GPUs in the same cluster is, and will remain, mutually exclusive

■ The unit of request stops being a count and becomes an object you can name, share, and select.

AFTER · DRA — two separate objects

1 ResourceClaimTemplate

```
kind: ResourceClaimTemplate
metadata:
  name: single-gpu
spec:
  spec:
    devices:
      requests:
        - name: gpu
          exactly:
            deviceClassName: gpu.nvidia.com
```

↓ referenced by name — one claim per pod

2 Pod spec · two containers, one shared claim

```
spec:
  containers:
    - name: ctr0
      resources: {claims: [{name: shared-gpu}]}
    - name: ctr1
      resources: {claims: [{name: shared-gpu}]}
  resourceClaims:
    - name: shared-gpu
      resourceClaimTemplateName: single-gpu
```

From one integer to every GPU shape



China 2026

Full GPU



isolation
none

`gpu.nvidia.com`

MIG slice



isolation
hardware

`mig.nvidia.com`

Time-slicing



one at a time

isolation
software

`TimeSlicingSettings`

MPS

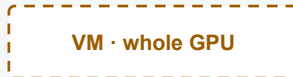


all at once

isolation
software, concurrent

`MPSSupport`

Passthrough



isolation
whole device + NVLink

`vfiio.gpu.nvidia.com`

CEL SELECTOR · narrow the device inside the claim

```
selectors:  
- cel:  
  expression: |  
    device.attributes["gpu.nvidia.com"].productName.lowerAscii().matches  
    ('^.*a100.*$')
```

WHAT THIS REPLACES

Under the device plugin, sharing was a node-level config file and selection was a node label. Both now live in the claim, next to the workload.

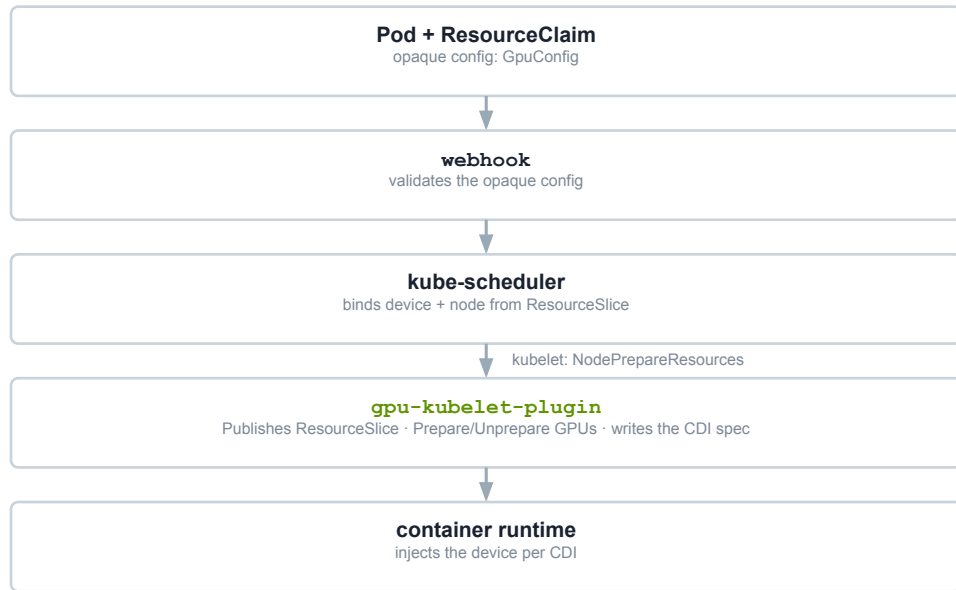
v0.5.0 · Adds multi-user MPS, Fabric Manager partitioning for passthrough isolation, and ConsumableShares for GPU sharing across namespaces — all alpha, off by default

■ Sharing is software, isolation is hardware, passthrough hands the driver back — all through one claim.

How a GPU request actually flows



China 2026



KUBERNETES OWNS

- the scheduler picks the device and the node from published ResourceSlices
- binding, retry and quota are core DRA, not driver code
- the claim with a ResourceClaimTemplate lifecycle follows the pod

THE DRIVER OWNS

- publishing device facts as ResourceSlices
- validating opaque config in the webhook
- Prepare: write the CDI spec · Unprepare: reset sharing config

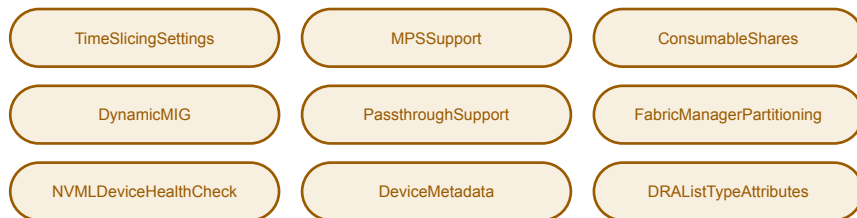
■ The driver publishes device facts and injects devices; every scheduling decision stays in core Kubernetes.

GPU feature gates and deployment

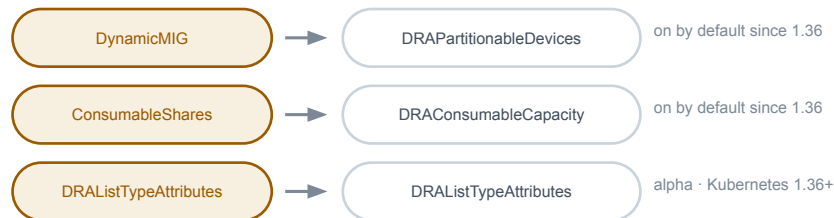


China 2026

GPU GATES · ALL ALPHA, ALL OFF BY DEFAULT



NEED AN UPSTREAM KUBERNETES GATE



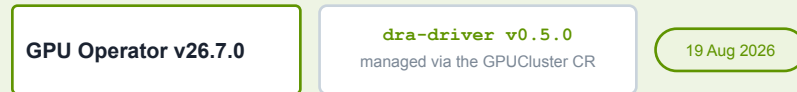
DEPLOYMENT · FROM SEPARATE CHART TO MANAGED COMPONENT

BEFORE · STANDALONE CHART



two installs to keep in sync · GPU Operator v25.10+ supplies CDI and driver 580+ · still supported; GPU allocation needs `gpuResourcesEnabledOverride=true`

TODAY · SHIPPED IN v0.5.0 + GPU OPERATOR v26.7.0



GPUCluster deploys dra-driver, ComputeDomains, DCGM and a DRA validator · no separate chart any more · GPUCluster and ClusterPolicy cannot coexist

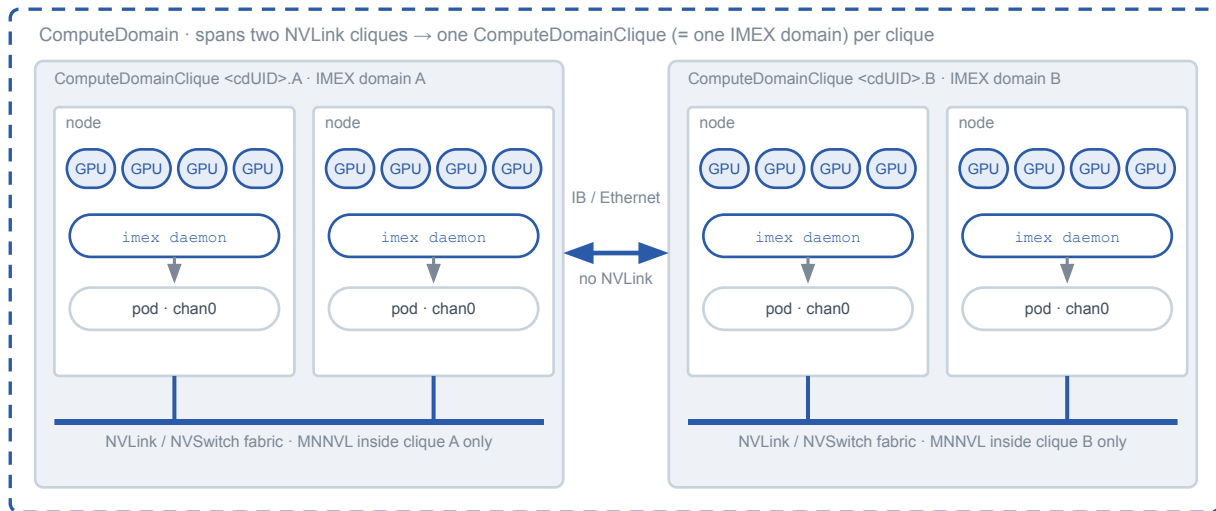
- From v0.5.0 the GPU Operator ships the driver.

ComputeDomain



China 2026

Part 2



compute-domain.nvidia.com

a second, independent subsystem in the same chart

```
DeviceClasses: compute-domain-daemon ·  
compute-domain-default-channel
```

officially supported

DESIGN GOAL

Make IMEX an implementation detail that neither workload authors nor cluster operators have to think about.

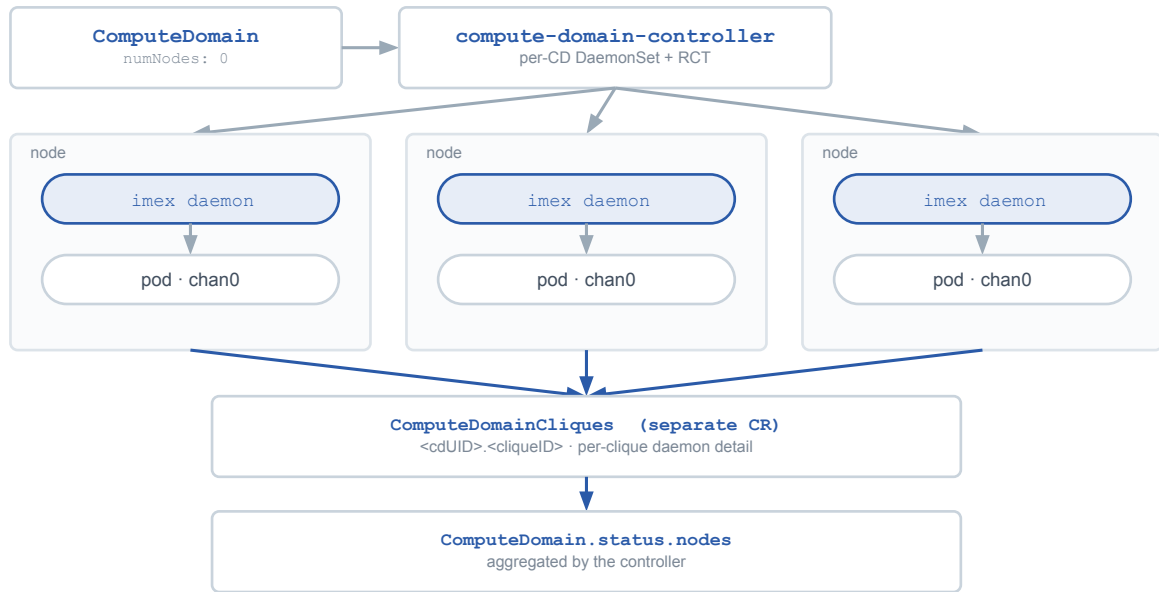
WHY IMEX · IMEX is what makes GPU memory shareable across node boundaries inside a domain — the driver orchestrates its daemons, domains and channels

■ The ComputeDomain is the abstraction; IMEX daemons, domains and channels are the implementation.

Driver-managed IMEX: the default workflow



China 2026



PROS

- zero host configuration — nothing to pre-install per domain
- domain membership follows the workload automatically
- daemon lifecycle is tied to the CD, so cleanup is automatic

CONS

- daemon bootstrap sits on the job-start path
- index negotiation at scale is the bottleneck
- host `nvidia-imex.service` must be disabled and masked

■ The driver owns everything — which is why the cost shows up at job-start time.

Host-managed IMEX: the alternative mode



China 2026

imex.mode = hostManaged · gate HostManagedIMEXDaemon · new in v0.5.0

ADMIN OWNS

nvidia-imex

nodes_config.cfg

config.cfg

systemd

the domain is defined on the host, before any workload exists



DRIVER OWNS

IMEX channel allocation only

nvidia-imex-ctl -q --u=<socket>

controller skips the per-CD DaemonSet and the daemon ResourceClaimTemplate

no daemon bootstrap on the job path

ComputeDomainClique tracking and DNS daemon naming are NOT used in this mode

PROS

- no per-CD daemon, so no index negotiation in the driver
- daemons are already running when the job arrives

CONS

- admin owns the IMEX node list on every node
- domain-level isolation only — channel 0 for everyone
- imex.mode / imex.isolation cannot change while CD workloads run

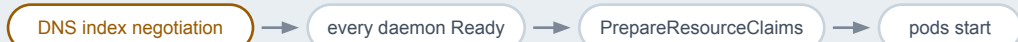
■ The coordination does not disappear — it moves from the driver to your configuration management.

Optimization at scale



China 2026

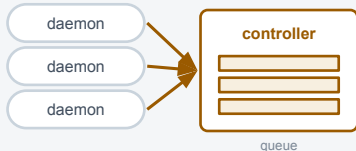
BASLINE · DISTRIBUTED, ONE SHARED COMPUTEDOMAIN OBJECT



every daemon optimistic-locks the same object to claim an index — 409 conflicts, and 429 APF throttling with Retry-After: 30s

PR #810

Centralized

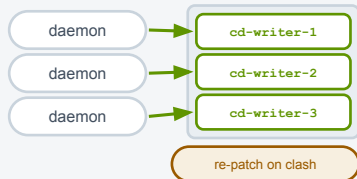


workqueue backlog

#816: a 256-pod job took several minutes to start

PR #822

Distributed + SSA

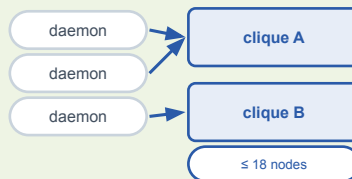


retries remain

significant apiserver performance regression in large cluster

PR #826

ComputeDomainClique

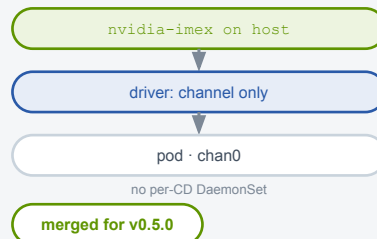


shipped · default on

300+ nodes ≈ 3s

PR #1236 · v0.5.0

Host-managed IMEX



merged for v0.5.0

admin owns nodes_config.cfg; domain isolation only

SHIPPED · #826 is ComputeDomainCliques, Beta and default on in v0.4.1 · #1236 merged for the v0.5.0 milestone

■ **v0.5.0 lets you skip it entirely.**

ComputeDomain challenges



China 2026

CLOSED — AND ON BY DEFAULT

Fabric errors

crash loudly instead of degrading silently

`CrashOnNVLinkFabricErrors`

IP vs DNS

stable names survive pod restarts

`IMEXDaemonsWithDNSNames`

Index negotiation

scoped to a single NVLink clique

`ComputeDomainCliques`

nothing to enable, nothing to tune — these are simply on

STILL OPEN

IMEX node list

In host-managed mode you own the IMEX node list on every node, and it has to match the domain you intended.

no automation yet

Driver-managed avoids it entirely — that is the trade you make for a faster start.

- Three are closed and on by default; the only open one is the list you maintain yourself.

Roadmap



China 2026

GPU

Support for NVIDIA vGPUs

Bring virtualized GPUs into the same DeviceClass and claim model.

GPU

Graduate core GPU features to beta

Enabled by default — no feature gate to flip before you can use them.

GPU

MPS compatibility with Consumable Shares

Implement compatibility between MPS and Consumable Shares

GPU

Per-node configuration

Set feature gates and GPU sharing config per node rather than per cluster.

COMPUTEDOMAIN

ComputeDomain / IMEX improvements

A boundary beyond a single NVLink partition, and long-running cluster-wide daemons.

COMPUTEDOMAIN

Workload API integration

Tie a ComputeDomain to the workload object that owns it, not just to its pods.

Call for actions



China 2026

DRA is in scope of Kubernetes WG Device Management.

- Weekly Meetings
 - Main WG Meeting
 - Biweekly Tuesdays 8:30am PST
 - Biweekly Wednesdays 9:00am CET
 - Native Resources in DRA Meeting
 - Biweekly Tuesdays 8:30am PST
 - [Zoom](#)
 - [Agenda](#)
 - [Playlist](#)
- Very active [Slack](#) channel: #wg-device-management
- Less active [Mailing](#) List
- [Community Page](#)

NVIDIA GPU DRA Driver

- Slack Channels
 - #sig-node
 - #gpu-nvidia
 - #wg-device-management
- [Mailing](#) List



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



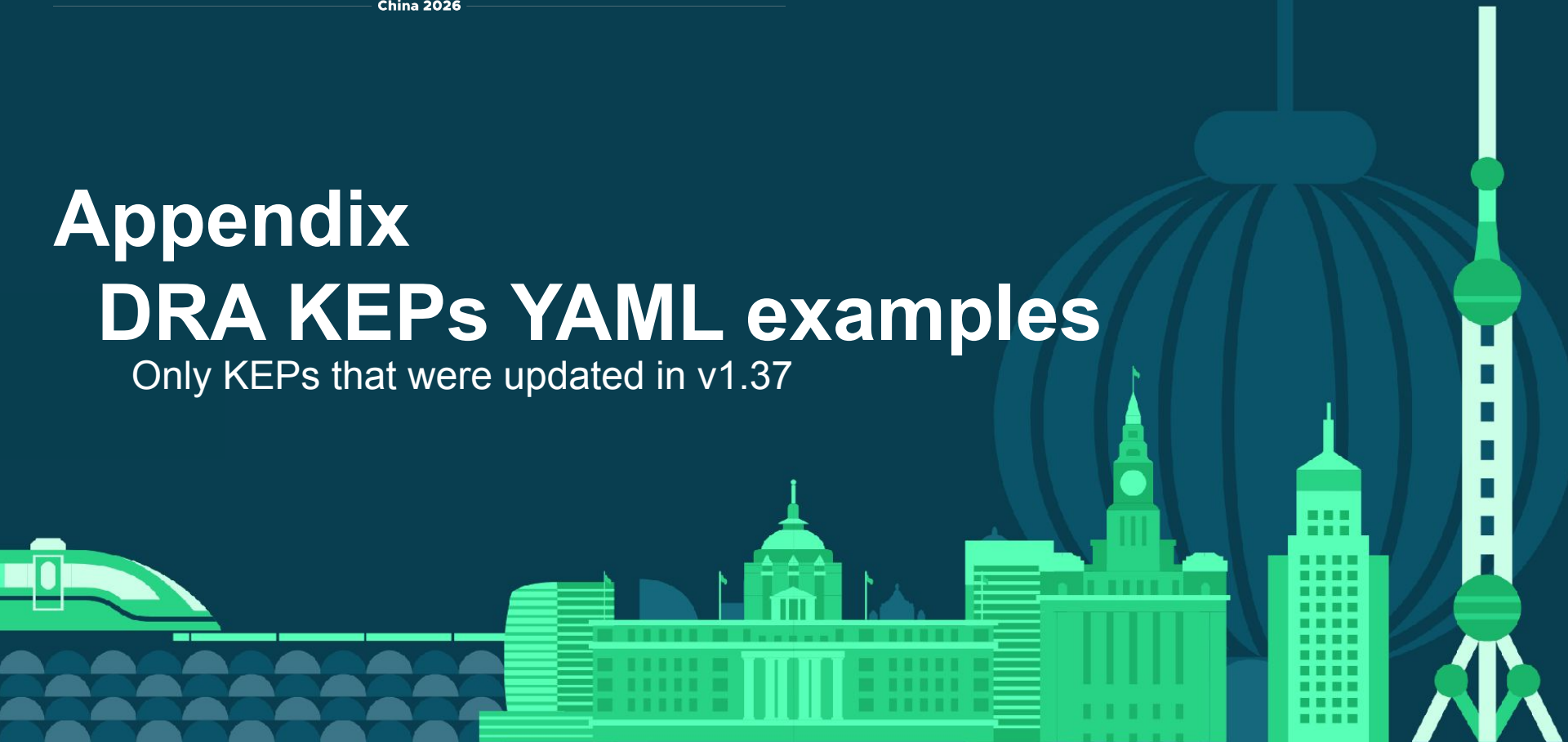
PyTorch
CONFERENCE

China 2026

Appendix

DRA KEPs YAML examples

Only KEPs that were updated in v1.37



KEP-5004 · 调度与兼容



China 2026

Extended Resource 请求由 DRA 驱动接管

Stable

Pod 仍写传统 limits; DeviceClass 把资源名映射到 DRA, 请求映射落到 Pod status。

1 DeviceClass 发布映射

```
apiVersion: resource.k8s.io/v1
kind: DeviceClass
metadata:
  name: gpu.example.com
spec:
  extendedResourceName: example.com/gpu
  selectors:
  - cel:
      expression: |
        device.driver ==
        "gpu.example.com"
```

2 Pod 使用旧式 资源请求

```
apiVersion: v1
kind: Pod
metadata:
  name: trainer
spec:
  containers:
  - name: train
    image: example/train:v1
    resources:
      limits:
        example.com/gpu: 1
      requests:
        example.com/gpu: 1
```

3 系统生成 Claim + Pod status

```
status:
  extendedResourceClaimStatus:
    resourceClaimName:
      trainer-gpu-7f9d
    requestMappings:
    - containerName: train
      resourceName:
        example.com/gpu
      requestName: req-0
  ---
# ResourceClaim 已分配
status:
  allocation:
    devices:
      results:
      - request: req-0
        driver: gpu.example.com
        pool: node-a
        device: gpu-0
```

闭环结果

应用无需改成 ResourceClaim API; 调度器创建/分配隐式 Claim, Pod status 给出容器→资源名→request 的可追踪映射。

v1.37 Stable
DRAExtendedResource (GA, 锁定开启)

Extended Resource 请求由 DRA 驱动接管



China 2026

Pod 仍写传统 limits; DeviceClass 把资源名映射到 DRA, 请求映射落到 Pod status。

Stable

1 DeviceClass 发布映射

```
apiVersion: resource.k8s.io/v1
kind: DeviceClass
metadata:
  name: gpu.example.com
spec:
  extendedResourceName: example.com/gpu
  selectors:
  - cel:
      expression: |
        device.driver ==
        "gpu.example.com"
```

2 Pod 使用旧式 资源请求

```
apiVersion: v1
kind: Pod
metadata:
  name: trainer
spec:
  containers:
  - name: train
    image: example/train:v1
    resources:
      limits:
        example.com/gpu: 1
      requests:
        example.com/gpu: 1
```

3 系统生成 Claim + Pod status

```
status:
  extendedResourceClaimStatus:
    resourceClaimName:
      trainer-gpu-7f9d
    requestMappings:
    - containerName: train
      resourceName:
        example.com/gpu
      requestName: req-0
---
# ResourceClaim 已分配
status:
  allocation:
    devices:
      results:
      - request: req-0
        driver: gpu.example.com
        pool: node-a
        device: gpu-0
```

闭环结果

应用无需改成 ResourceClaim API; 调度器创建/分配隐式 Claim, Pod status 给出容器→资源名→request 的可追踪映射。

v1.37 Stable
DRAExtendedResource (GA, 锁定开启)

KEP-4817 · 状态与可观测性



China 2026

驱动把已分配设备的运行状态写回 ResourceClaim

allocation 表示“分了什么”; status.devices 表示“设备现在怎样、网络信息是什么”。

Stable

1 分配结果: 设备身份

```
apiVersion: resource.k8s.io/v1
kind: ResourceClaim
metadata:
  name: sriov-nic
status:
  allocation:
    devices:
      results:
        - request: nic
          driver: sriov.example.com
          pool: node-a-pf0
          device: vf-7
          shareID: pod-a
```

2 驱动写入设备状态

```
status:
  devices:
    - driver: sriov.example.com
      pool: node-a-pf0
      device: vf-7
      shareID: pod-a
      conditions:
        - type: Ready
          status: "True"
          reason: Configured
      networkData:
        interfaceName: net1
        ips:
          - 10.42.0.27/24
        hardwareAddress:
          02:00:00:00:00:07
      data:
        migUUID: MIG-...
```

3 故障时的可观测变化

```
status:
  devices:
    - driver: sriov.example.com
      pool: node-a-pf0
      device: vf-7
      shareID: pod-a
      conditions:
        - type: Ready
          status: "False"
          reason: LinkDown
          message: |
            PF carrier is down
      networkData:
        interfaceName: net1
        ips:
          - 10.42.0.27/24
```

闭环结果

status.devices 的 driver/pool/device/shareID 必须与 allocation result 对齐; 消费者可直接判断 Ready, 并读取标准化网卡信息。

v1.37 Stable
DRAResourceClaimDeviceStatus (GA)

设备 taint 在调度前阻止使用, 也可 驱逐现有使用者

管理员对具体 GPU 加 NoExecute; 只有声明匹配 toleration 的 Claim 能继续使用。

Stable

1 管理员标记维护设备

```
apiVersion: resource.k8s.io/v1
kind: DeviceTaintRule
metadata:
  name: gpu-0-maintenance
spec:
  deviceSelector:
    driver: gpu.example.com
    pool: node-a
    device: gpu-0
  taint:
    key: maintenance.example.com
    value: firmware
    effect: NoExecute
    timeAdded: 2026-08-18T02:00:00Z
```

2 关键任务容忍该 taint

```
apiVersion: resource.k8s.io/v1
kind: ResourceClaim
metadata:
  name: critical-gpu
spec:
  devices:
    requests:
      - name: gpu
        exactly:
          deviceClassName:
            gpu.example.com
          tolerations:
            - key: maintenance.example.com
              operator: Equal
              value: firmware
              effect: NoExecute
              tolerationSeconds: 3600
```

3 结果与执行状态

```
status:
  allocation:
    devices:
      results:
        - request: gpu
          driver: gpu.example.com
          pool: node-a
          device: gpu-0
          tolerations:
            - key: maintenance.example.com
              operator: Equal
              value: firmware
              effect: NoExecute
---
# DeviceTaintRule
status:
  conditions:
    - type: EvictionInProgress
      status: "False"
```

闭环结果

无 toleration 的新 Claim 不会选中 gpu-0; 既有 Pod 可被 NoExecute 驱逐。示例 Claim 最多容忍 1 小时, 结果中保留实际 toleration。

v1.37 Stable
DRADeviceTaints /
DRADeviceTaintRules (GA)

KEP-6072 · 拓扑与 NUMA



China 2026

用标准 resource.kubernetes.io/numaNode 跨驱动做 NUMA 共置

GPU 与 NIC 由不同驱动发布, 但共享同一个标准 list 属性, constraint 取交集。

Stable

1 两个驱动发布标准属性

```
apiVersion: resource.k8s.io/v1
kind: ResourceSlice
metadata: {name: node-a-gpu}
spec:
  driver: gpu.example.com
  pool: {name: node-a, generation: 1,
resourceSliceCount: 1}
  nodeName: node-a
  devices:
  - name: gpu-0
    basic:
      attributes:
        resource.kubernetes.io/numaNode: {ints: [0]}
---
apiVersion: resource.k8s.io/v1
kind: ResourceSlice
metadata: {name: node-a-nic}
spec:
  driver: nic.example.com
  pool: {name: node-a, generation: 1,
resourceSliceCount: 1}
  nodeName: node-a
  devices:
  - name: nic-3
    basic:
      attributes:
        resource.kubernetes.io/numaNode: {ints: [0, 1]}
  - name: nic-4
    basic:
      attributes:
        resource.kubernetes.io/numaNode: {ints: [1]}
```

2 Claim 要求 GPU/NIC 共 NUMA

```
apiVersion: resource.k8s.io/v1
kind: ResourceClaim
metadata:
  name: gpu-and-nic
spec:
  devices:
    requests:
    - name: gpu
      exactly:
        deviceClassName: gpu.example.com
    - name: nic
      exactly:
        deviceClassName: nic.example.com
  constraints:
  - requests: [gpu, nic]
    matchAttribute:
      resource.kubernetes.io/numaNode
```

3 调度选择属性有交集的一组

```
status:
  allocation:
    devices:
      results:
      - request: gpu
        driver: gpu.example.com
        pool: node-a
        device: gpu-0
      - request: nic
        driver: nic.example.com
        pool: node-a
        device: nic-3
---
# 交集: gpu-0 {0} ∩ nic-3 {0,1}
#       = {0}
# nic-4 只有 {1}, 不能与 gpu-0 配对
```

闭环结果

matchAttribute 比较 list 的交集, 不要求完整相等; 最终 status 只记录设备身份, numaNode 仍以 ResourceSlice 为准。

v1.37 Stable
无独立 gate; list 属性依赖
DRAListTypeAttributes

KEP-5304 · 工作负载可见性



China 2026

把已分配设备属性与驱动状态投影为容器内 metadata 文件

应用不需要调用 Kubernetes API; kubelet 为每个 claim/request 生成只读设备元数据。

Beta

1 Slice + Claim status 提供来源

```
apiVersion: resource.k8s.io/v1
kind: ResourceSlice
metadata: {name: node-a-nic}
spec:
  driver: nic.example.com
  pool: {name: node-a, generation: 1,
resourceSliceCount: 1}
  nodeName: node-a
  devices:
  - name: nic-7
    basic:
      attributes:
        resource.kubernetes.io/pciBusID: {string:
"0000:65:00:1"}
---
apiVersion: resource.k8s.io/v1
kind: ResourceClaim
metadata: {name: fast-nic}
status:
  devices:
  - driver: nic.example.com
    pool: node-a
    device: nic-7
    networkData:
      interfaceName: net1
      ips: ["10.42.0.27/24"]
    data:
      migUUID: MIG-...
```

2 Pod 使用这个 Claim

```
apiVersion: v1
kind: Pod
metadata:
  name: net-app
spec:
  resourceClaims:
  - name: fast-nic
    resourceClaimName: fast-nic
  containers:
  - name: app
    image: example/net-app:v2
    resources:
      claims:
      - name: fast-nic
      command: ["cat"]
      args:
      -
/var/run/kubernetes.io/dra-device-attributes/fast-
nic/nic/metadata.json
```

3 容器内 metadata.json

```
{
  "apiVersion": "metadata.resource.k8s.io/v1beta1",
  "kind": "DeviceMetadata",
  "metadata": {
    "name": "fast-nic",
    "namespace": "default"
  },
  "podClaimName": "fast-nic",
  "requests": [
    {
      "name": "nic",
      "devices": [
        {
          "driver": "nic.example.com",
          "pool": "node-a",
          "name": "nic-7",
          "attributes": {
            "resource.kubernetes.io/pciBusID": {
              "string": "0000:65:00:1"
            },
            "nic.example.com/migUUID": {
              "string": "MIG-..."
            }
          }
        }
      ]
    }
  ],
  "networkData": {
    "interfaceName": "net1",
    "ips": ["10.42.0.27/24"]
  }
}
```

闭环结果

metadata 文件把 Slice 属性与 Claim 的 driver-owned status 合并; 路径按 claimName/requestName 稳定组织, 可由普通进程读取。

v1.37 Beta
无 kube feature gate; 驱动
EnableDeviceMetadata

KEP-5729 · 工作负载级共享



China 2026

PodGroup 中多个 Pod 共享一份模板化 ResourceClaim

PodGroup 创建 Claim; 每个成员 Pod 用同名逻辑引用, status 解析到同一个实际 Claim。

Beta

1 PodGroup 声明共享模板

```
apiVersion: scheduling.k8s.io/v1beta1
kind: PodGroup
metadata:
  name: train-job
spec:
  resourceClaims:
  - name: model-cache
    resourceClaimTemplateName:
      model-cache-template
  minMember: 2
```

2 成员 Pod 引用逻辑名称

```
apiVersion: v1
kind: Pod
metadata:
  name: worker-0
spec:
  schedulingGroup:
    podGroupName: train-job
  resourceClaims:
  - name: model-cache
    resourceClaimTemplateName:
      model-cache-template
  containers:
  - name: worker
    image: example/train:v4
    resources:
      claims:
      - name: model-cache
  ---
# worker-1 使用相同声明
```

3 Group / Pod / Claim 三方闭环

```
# PodGroup status
resourceClaimStatuses:
- name: model-cache
  resourceClaimName:
    train-job-model-cache-7db2
---
# Pod status
resourceClaimStatuses:
- name: model-cache
  resourceClaimName:
    train-job-model-cache-7db2
---
# ResourceClaim status
reservedFor:
- apiGroup: scheduling.k8s.io
  resource: podgroups
  name: train-job
  uid: 0d9c...
```

闭环结果

实际 Claim 由 PodGroup 生命周期管理, Pod 只持有逻辑名; PodGroup/Pod status 与 Claim.reservedFor 能互相核对。

v1.37 Beta
DRAWorkloadResourceClaims +
GenericWorkload(Beta)

KEP-5517 · 核心资源计费



China 2026

DRA 设备分配可以同步消耗 节点 CPU / memory allocatable

设备声明 resource 映射或固定开销; 调度结果把最终计费量写回 Pod status。

Alpha 2

1 设备声明 CPU 映射与内存开销

```
apiVersion: resource.k8s.io/v1
kind: ResourceSlice
metadata:
  name: node-a-accelerator
spec:
  driver: accelerator.example.com
  pool:
    name: node-a
    generation: 1
    resourceSliceCount: 1
  nodeName: node-a
  devices:
  - name: accelerator-0
    basic:
      nodeAllocatableResources:
        cpu:
          mapping:
            deviceMultiplier: "4"
        memory:
          overhead:
            perPod: 1Gi
            perContainer: 512Mi
```

2 Pod 同时请求设备和 CPU

```
apiVersion: v1
kind: Pod
metadata:
  name: inference
spec:
  resourceClaims:
  - name: accelerator
    resourceClaimName: accelerator
  containers:
  - name: server
    image: example/infer:v3
    resources:
      requests:
        cpu: "2"
        memory: 2Gi
      claims:
      - name: accelerator
```

3 Pod status 显示额外计费

```
status:
  nodeAllocatableResourceClaimStatuses:
  - resourceClaimName: accelerator
    containers:
    - server
      mapping:
        - name: cpu
          quantity: "4"
      overhead:
        - name: memory
          perPod: 1Gi
          perContainer: 512Mi
```

闭环结果

调度时不只验证设备, 还把 CPU 4 核与 memory 开销纳入节点可分配量; status 展示设备导致的额外扣减, 避免隐性超卖。

v1.37 Alpha 2
DRANodeAllocatableResources (Alpha,
默认关闭)

KEP-5677 · 容量可见性



China 2026

用 ResourcePoolStatusRequest 查询池的可用、已分配与不可用 设备

控制器汇总 ResourceSlice 与 Claim; 自动扩缩容器无需枚举每个设备对象。

Alpha

1 池由多个 Slice 发布

```
apiVersion: resource.k8s.io/v1
kind: ResourceSlice
metadata:
  name: node-a-gpu-1
spec:
  driver: gpu.example.com
  pool:
    name: node-a-gpus
    generation: 7
    resourceSliceCount: 2
  devices:
  - name: gpu-0
    basic: {}
  - name: gpu-1
    basic: {}
```

2 请求池级摘要

```
apiVersion: resource.k8s.io/v1alpha3
kind: ResourcePoolStatusRequest
metadata:
  name: gpu-capacity
spec:
  driver: gpu.example.com
  poolName: node-a-gpus
  limit: 10
```

3 聚合后的 availability status

```
status:
  poolCount: 1
  pools:
  - driver: gpu.example.com
    poolName: node-a-gpus
    generation: 7
    resourceSliceCount: 2
    nodeName: node-a
    totalDevices: 8
    allocatedDevices: 5
    availableDevices: 2
    unavailableDevices: 1
  conditions:
  - type: Complete
    status: "True"
    reason: Processed
    message: Pool status calculated
```

闭环结果

status 是快照而非预约; poolCount/limit 能发现截断, Complete=True 表示本次聚合完成, autoscaler 可根据 availableDevices 决策。

v1.37 Alpha
DRResourcePoolStatus (Alpha, 默认关闭)

KEP-5945 · 节点准备



China 2026

远程/纯配置设备可以声明跳过 kubelet 节点侧操作

Alpha

驱动在 ResourceSlice 上声明 skipNodeOperations; 分配结果复制该决策, kubelet 不调用 prepare/unprepare。

1 远程设备无需节点插件

```
apiVersion: resource.k8s.io/v1
kind: ResourceSlice
metadata:
  name: remote-fpga
spec:
  driver: fpga.example.com
  pool:
    name: remote-pool
    generation: 3
    resourceSliceCount: 1
  skipNodeOperations:
    - ""
  devices:
    - name: fpga-a
      basic:
        attributes:
          fpga.example.com/endpoint:
            string: grpc://fpga-a:7443
```

2 普通 Claim 请求设备

```
apiVersion: resource.k8s.io/v1
kind: ResourceClaim
metadata:
  name: remote-fpga
spec:
  devices:
    requests:
      - name: fpga
        exactly:
          deviceClassName:
            fpga.example.com
        count: 1
```

3 结果携带跳过操作声明

```
status:
  allocation:
    devices:
      results:
        - request: fpga
          driver: fpga.example.com
          pool: remote-pool
          device: fpga-a
          skipNodeOperations:
            - ""
---
# kubelet 行为
# ✓ 不调用 NodePrepareResources
# ✓ 不调用 NodeUnprepareResources
# ✓ 容器直接使用远程 endpoint
```

闭环结果

** 同时跳过 prepare/unprepare; 若只声明 NodePrepareResources, 必须也跳过 NodeUnprepareResources。节点需声明支持该 feature。

v1.37 Alpha
DRAOptionalNodeOperations +
NodeDeclaredFeatures (Alpha)

KEP-5963 · 调度与绑定



China 2026

同一物理 GPU 的 MIG 与 vGPU 分区不能混合共分配

两个 compatibilityGroups 共享 counters; 调度器只接受交集非空的组合, 并在绑定前完成拒绝。

Alpha

1 ResourceSlice: MIG + vGPU 两组

```
apiVersion: resource.k8s.io/v1
kind: ResourceSlice
metadata:
  name: node-a-gpu-0
spec:
  driver: gpu.example.com
  pool: {name: node-a, generation: 4,
resourceSliceCount: 1}
  sharedCounters:
    - name: gpu-0-counters
      counters:
        multiprocessors: {value: "100"}
  devices:
    - name: gpu-0-mig-1g-0
      consumesCounters:
        counterSet: gpu-0-counters
        counters: {multiprocessors: {value: "20"}}
        compatibilityGroups: [mig]
    - name: gpu-0-mig-1g-1
      consumesCounters:
        counterSet: gpu-0-counters
        counters: {multiprocessors: {value: "20"}}
        compatibilityGroups: [mig]
    - name: gpu-0-vgpu-0
      consumesCounters:
        counterSet: gpu-0-counters
        counters: {multiprocessors: {value: "50"}}
        compatibilityGroups: [vgpu]
```

2 Claim: 请求同池 2 个逻辑 GPU

```
apiVersion: resource.k8s.io/v1
kind: ResourceClaim
metadata:
  name: two-gpus
spec:
  devices:
    requests:
      - name: gpu
        exactly:
          deviceClassName:
            gpu.example.com
          count: 2
  # 场景中三个逻辑设备共享
  # gpu-0-counters; 容量允许
  # MIG+MIG 或 MIG+vGPU
```

3 绑定后 status: 选择两个 MIG

```
status:
  allocation:
    nodeSelector:
      nodeSelectorTerms:
        - matchFields:
            - key: metadata.name
              operator: In
              values: [node-a]
    devices:
      results:
        - request: gpu
          driver: gpu.example.com
          pool: node-a
          device: gpu-0-mig-1g-0
        - request: gpu
          driver: gpu.example.com
          pool: node-a
          device: gpu-0-mig-1g-1
  # {mig} ∩ {mig} = {mig} ✓
  # {mig} ∩ {vgpu} = ∅ → Filter 阶段拒绝
```

闭环结果

成功绑定的 status 只记录最终设备, 不复制 compatibilityGroups; MIG+vGPU 即使 counters 容量足够也会在调度阶段失败, 不再拖到节点 prepare。

v1.37 Alpha
DRADeviceCompatibilityGroups +
DRAPartitionableDevices

Derived Attributes 在 Claim 内统一不同厂商的属性命名

GPU 写 numa、NIC 写 numaNode; 每个 request 用 CEL 派生 shared-numa-node, 再用 matchAttribute 共置。

Alpha

1 不同驱动、不同属性 schema

```
apiVersion: resource.k8s.io/v1
kind: ResourceSlice
metadata: {name: node-a-gpu}
spec:
  driver: gpu.nvidia.com
  pool: {name: node-a, generation: 1,
resourceSliceCount: 1}
  nodeName: node-a
  devices:
    - name: gpu-0
      basic:
        attributes:
          gpu.nvidia.com/numa: {int: 0}
---
apiVersion: resource.k8s.io/v1
kind: ResourceSlice
metadata: {name: node-a-nic}
spec:
  driver: dra.net
  pool: {name: node-a, generation: 1,
resourceSliceCount: 1}
  nodeName: node-a
  devices:
    - name: nic-3
      basic:
        attributes:
          dra.net/numaNode: {int: 0}
    - name: nic-4
      basic:
        attributes:
          dra.net/numaNode: {int: 1}
```

2 每个 request 派生统一属性

```
spec:
  devices:
    requests:
      - name: gpu
        exactly:
          deviceClassName: gpu.example.com
          derivedAttributes:
            - name: shared-numa-node
              expression: |
                device.attributes[
                  "gpu.nvidia.com"].numa
      - name: nic
        exactly:
          deviceClassName: nic.example.com
          derivedAttributes:
            - name: shared-numa-node
              expression: |
                device.attributes[
                  "dra.net"].numaNode
  constraints:
    - requests: [gpu, nic]
      matchAttribute: shared-numa-node
```

3 status 记录同 NUMA 的设备

```
status:
  allocation:
    devices:
      results:
        - request: gpu
          driver: gpu.example.com
          pool: node-a
          device: gpu-0
        - request: nic
          driver: nic.example.com
          pool: node-a
          device: nic-3
---
# 派生值仅参与调度:
# gpu-0 → shared-numa-node = 0
# nic-3 → shared-numa-node = 0
# nic-4 → shared-numa-node = 1(不选)
```

闭环结果

派生属性只在本次 allocation 计算中存在, 不写回 ResourceSlice/Claim status; 它解决跨厂商 schema 对齐, 不改变驱动的属性。

v1.37 Alpha
DRADerivedAttributes (Alpha, 默认关闭)



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

China 2026

Thanks

